

URL Shortener & Link Analytics

Welcome — and thanks for taking the time. This is a short, practical build exercise. We're not looking for a finished product; we're interested in how you think, structure code, and make decisions.

Using AI is encouraged. You may — and we expect you to — use Claude, Claude Code, or any AI assistant throughout. We're not testing whether you can write code from memory. We care about how you direct the tool, review what it produces, and own the result. Please build something you understand well enough to explain and extend in a follow-up live session.

The assignment

Build a small service that turns long URLs into short codes and redirects visitors to the original link when they hit a short code. It's a familiar problem on the surface — we're interested in the design decisions underneath.

Your service should provide:

- **POST /shorten** — accept a URL and return a short code.
- **GET /{code}** — redirect (301) to the original URL.
- Persist the mappings in a datastore of your choice.
- Support custom aliases, and return a 404 for unknown codes.
- Design a short-code generator that won't collide, and be ready to explain why it won't.
- Validate incoming URLs and decide (intentionally) how you handle the same URL being shortened twice.

What a strong submission demonstrates

- Shorten → redirect round-trips correctly, and an unknown code returns 404.
- Your duplicate-URL and custom-alias behaviour is deliberate and documented.
- Short codes are URL-safe and don't collide across repeated calls.

What to submit

- **A git repository** with its full commit history (please don't squash everything into one commit — we like to see how it came together).
- **A README** with clear instructions to install, run, and test the project.
- **Automated tests** covering the core logic and a few edge cases.
- **A one-page write-up** (template below) — this is required and is one of the most important parts.

The write-up (required)

In about one page, please answer:

1. What did you ask the AI to do, and what did you write or decide yourself?
2. Where did you override, correct, or throw away the AI's output — and why?
3. The two or three biggest trade-offs you made, and the alternatives you considered.
4. What's missing, or what you'd do with another day?

How we evaluate

We read the write-up and commit history first, then run and read the code. We value working software, a clean data model, meaningful tests, and — above all — clear evidence of your judgment: how you used the AI, where you steered or corrected it, and the trade-offs you chose.

Time & logistics

- **Effort:** aim for roughly 3–4 focused hours. It's fine to leave things unfinished — just note them in the write-up.
- **Return window:** your recruiter will confirm the deadline.
- **Follow-up:** if this goes well, the next round is a live session where you'll extend this same project with us — so please keep your repo and environment handy.
- **Stack:** use any language, framework, and datastore you're comfortable with.

A note on originality. Please treat this exercise as confidential — don't post it or your solution publicly. The work should be your own (with your AI assistant), and you should be able to walk us through every part of it.